TP2 — Mise en page d'un magazine

Module CSS avancé · Licence 3 · Jour 1 · 14h15 – 15h30

Objectifs pédagogiques

À l'issue de ce TP, vous devez être capable de :

1. **construire un composant en Flexbox** : raccourci `flex`, `gap`, `flex-wrap`, alignements, `margin-inline-start: auto` ;
2. **donner une hauteur** à un élément sans contenu avec **`aspect-ratio`** ;
3. **créer un gabarit de page à lignes nommées** (`[contenu-start]`…) et des bandeaux pleine largeur sans `max-width` ni `margin: auto` ;
4. **concevoir une mise en page mobile-first** avec **`grid-template-areas`**, puis la redessiner à chaque point de rupture ;
5. **utiliser le dimensionnement intrinsèque** : `max-content`, `min-content`, `fit-content`, `fit-content()` ;
6. **écrire une grille de cartes responsive sans media query**, avec `repeat(auto-fit, minmax(min(100%, …), 1fr))` ;
7. **aligner des cartes entre elles avec `subgrid`**.

Prérequis

- Cours 2 (Grid avancé, Flexbox revisité, dimensionnement intrinsèque, `aspect-ratio`).
- TP1 : couches et nesting natif. Le starter est déjà organisé en `@layer reset, base, layout, components` et vous écrivez en nesting.

Durée et organisation

Durée	1 h 15 (étapes 1 à 7 ; exercice d'approfondissement facultatif)
Organisation	en groupe de trois (trinôme). Alternez au clavier à chaque étape ; pendant ce temps, un membre vérifie la grille avec l'inspecteur Grid des DevTools et un autre teste la page à 320 px, 768 px et 1280 px. Chaque membre doit coder.
Fichiers	`starter/index.html` (ne pas modifier), `starter/style.css` (à compléter)
Navigateur	Chrome ou Edge récent. Ouvrez `index.html` par double-clic, puis **F12**, onglet **Éléments** : cliquez sur le badge `grid` ou `subgrid` à côté d'un élément pour afficher la grille et ses **noms de lignes et de zones**

Livrable et critères de réussite

- **Livrable** : le fichier `starter/style.css` complété, présenté à l'enseignant à 15h30 (fenêtre à 1280 px, puis à 320 px en mode responsive).

- **Critères de réussite** : la liste détaillée figure en fin d'énoncé. Servez-vous-en pour vous auto-évaluer au fil des étapes ; l'enseignant la vérifie en fin de séance. Ce TP relève de l'évaluation formative : il n'est pas noté.
- L'exercice d'approfondissement est **facultatif**.

Contexte de l'exercice

La rédaction du *Grand Format*, magazine étudiant de l'IUT, fournit le HTML, les couleurs et les polices de son site : il vous reste à réaliser **toute la mise en page** (grande Une, encadré de chiffres, citation pleine largeur, cartes alignées, sommaire). Le site étant surtout consulté sur téléphone, **aucun défilement horizontal n'est toléré à 320 px**.

Ouvrez `starter/index.html` : tout est empilé, les images sont des bandes de 6rem de haut, la navigation est une liste verticale. L'objectif est d'en faire une page de magazine.

Étape 1 — Le masthead en Flexbox (≈ 8 min)

Dans `.masthead` (bloc `TODO étape 1`) :

1. `.masthead__inner` : logo, navigation et bouton **sur une ligne**, centrés verticalement, avec un `gap`. Sur petit écran, les éléments **passent à la ligne** au lieu de déborder.
2. Le logo ne doit **jamais rétrécir** : utilisez le raccourci `flex`.
3. Les liens de `.masthead__liens` côte à côte, avec un écart, et qui passent à la ligne eux aussi.
4. Le bouton « S'abonner » est **poussé tout à droite**, sans `position` ni `justify-content: space-between`.

Indice 1 — pousser un élément

Dans un conteneur flex, une marge `auto` absorbe tout l'espace libre de son côté. `margin-inline-start: auto` sur le dernier élément le colle donc à droite (en français, de gauche à droite).

Indice 2 — le raccourci flex

Valeur	Équivaut à	Comportement
`flex: none`	`0 0 auto`	ne grandit pas, ne rétrécit pas : largeur `max-content`
`flex: auto`	`1 1 auto`	grandit et rétrécit à partir de sa taille de contenu
`flex: 1`	`1 1 0%`	se partage l'espace à parts égales, quel que soit le contenu

---

Étape 2 — Des visuels qui ont du corps : `aspect-ratio` (≈ 5 min)

Les « photos » sont des `<div role="img">` vides. Sans contenu, elles n'ont pas de hauteur, d'où la hauteur provisoire de 6rem dans la couche `base`.

1. Supprimez la ligne `min-block-size: 6rem` (bloc `.visuel`, `TODO étape 2`).

2. Le visuel de la Une est au format **4/3** sur mobile, et **21/9** à partir de `48rem`. Imbriquez la media query dans la règle.
3. Les visuels des cartes sont au format **3/2**.

Indice

`aspect-ratio: 3 / 2;` : le navigateur calcule la hauteur à partir de la largeur. C'est inutile sur un `<img>` qui a ses attributs `width` et `height` (il a déjà un ratio), mais indispensable pour un placeholder, un `<iframe>` vidéo ou un bloc décoratif.

Étape 3 — Le gabarit à lignes nommées (≈ 10 min)

Objectif : un contenu centré d'au plus `72rem` de large, avec au moins `1rem` de marge de chaque côté, **et** des bandeaux (masthead, citation, pied de page) dont le fond va d'un bord à l'autre de l'écran pendant que leur contenu reste aligné sur la colonne centrale.

1. Sur `.page`, définissez **3 colonnes** et **nommez les lignes** :

   ```
   [plein-start]  marge  [contenu-start]  contenu  [contenu-end]  marge  [plein-end]
   ```

   Chaque marge vaut au moins `var(--marge)` et se partage l'espace libre. Le contenu vaut au plus `var(--largeur-max)`.
2. `.contenu-principal` et `.pleine-largeur` occupent la zone `plein` et **réutilisent les colonnes de `.page`**.
3. Par défaut, **tous leurs enfants** vont dans la zone `contenu`.
4. Exception : la citation (`.pleine-largeur` placée **dans** `<main>`) va dans `plein`.

Vérifiez dans les DevTools (badge `subgrid` sur `<main>`) que les noms de lignes apparaissent bien.

Indice 1 — des lignes qui créent des zones

Deux lignes nommées `xxx-start` et `xxx-end` créent automatiquement une **zone implicite** `xxx`. On peut donc écrire `grid-column: contenu;` au lieu de `grid-column: contenu-start / contenu-end;`.

Indice 2 — la taille des colonnes

`minmax(var(--marge), 1fr)` pour les marges, `minmax(0, var(--largeur-max))` pour le contenu. Le `0` permet au contenu de rétrécir sous la largeur de son plus long mot sur un petit écran.

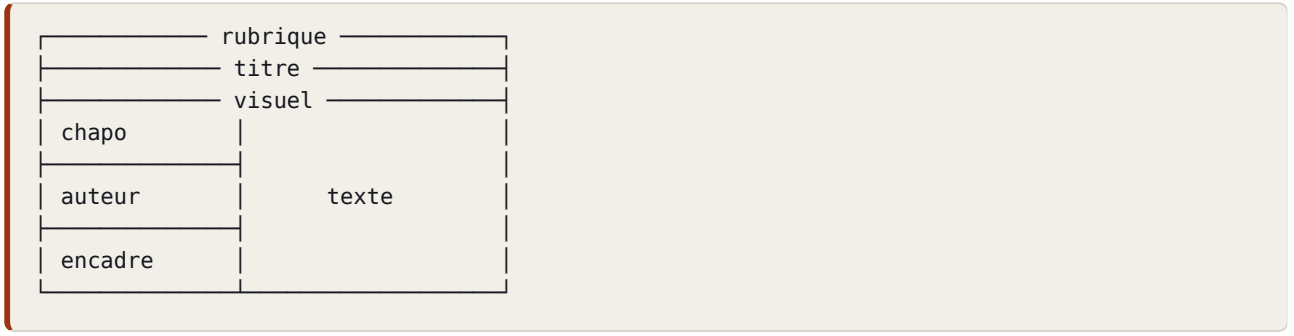
Indice 3 — l'exception de la citation

`.contenu-principal > *` met la citation dans `contenu`, et `.pleine-largeur` la met dans `plein`. Ces deux sélecteurs ont la même spécificité (0,1,0) et c'est le dernier écrit qui gagne : fragile. Écrivez une règle **plus spécifique** pour le cas « un `.pleine-largeur` enfant de `.contenu-principal` ».

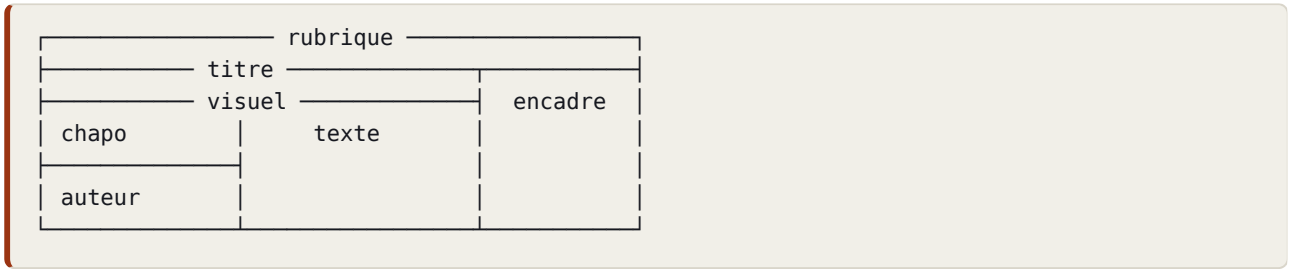
Étape 4 — La Une avec `grid-template-areas` (≈ 15 min)

Sept zones : `rubrique`, `titre`, `chapo`, `visuel`, `auteur`, `texte`, `encadre`.

1. Associez chaque enfant de `.une` à sa zone (nesting : `& .une__titre { grid-area: titre; }`…).
2. **Mobile** : une colonne, dans cet ordre : rubrique, titre, chapo, visuel, auteur, texte, encadre.
3. **À partir de 48rem** : 2 colonnes (`1fr` et `1.6fr`).



4. **À partir de 64rem** : l'encadré « En chiffres » passe dans une **3e colonne** à droite, à côté du titre et du visuel.



   Pour l'instant, donnez-lui une largeur fixe de `18rem` : vous l'améliorerez à l'étape 5.

Indice 1 — syntaxe

```
grid-template-areas:
  "rubrique rubrique"
  "titre    titre"
  "chapo    texte";
```

Une chaîne par ligne, un nom par colonne. Une zone doit être un **rectangle**. Un point `.` désigne une cellule vide. Alignez les noms en colonnes : vous voyez la mise en page dans le code.

Indice 2 — un texte très long étire tout

Si le texte est bien plus haut que le chapo et l'auteur, ces lignes s'étirent et laissent du vide. À 64rem, ajoutez `grid-template-rows: auto auto auto auto 1fr;` : seule la dernière ligne absorbe l'espace restant. Pensez aussi à `align-items: start`.

Indice 3 — et l'accessibilité ?

`grid-template-areas` change l'ordre **visuel**, pas l'ordre du HTML : le lecteur d'écran et la touche Tab suivent le HTML. Ici, l'ordre visuel reste proche de l'ordre du code. Gardez ce réflexe dans votre projet.

Étape 5 — Dimensionnement intrinsèque (≈ 10 min)

Plus de largeurs devinées « à l'œil » : c'est le contenu qui décide.

1. **Étiquettes** (`.tag`) : un `<p class="tag">` s'étire sur toute la largeur. Réduisez-le à la largeur de son texte.
2. **Sommaire** (`.sommaire` , une `<dl>`) : une grille à 2 colonnes. La 1re est **aussi large que le plus long numéro de page** (« p. 104 »), sans valeur fixe. La 2e prend le reste. Alignez les numéros à droite.
3. **Encadré « En chiffres »** : chaque ligne (`.chiffres > div`) affiche l'intitulé à gauche et le chiffre à droite, alignés sur la **ligne de base**. « 2 000 m² » ne passe jamais sur deux lignes.
4. **Colonne de l'encadré** (étape 4, à 64rem) : remplacez `18rem` par une taille qui suit **la largeur du contenu**, plafonnée à `18rem` .
5. **Mots-clés** (`.tags`) : côte à côte, avec passage à la ligne.

Indice — les mots-clés intrinsèques

Mot-clé	Largeur
`min-content`	la plus petite largeur sans débordement (le mot le plus long)
`max-content`	la largeur du contenu sur une seule ligne, sans retour
`fit-content`	`max-content` , mais sans dépasser la place disponible
`fit-content(18rem)` *(dans une grille)*	comme `fit-content` , plafonné à 18rem

Ils s'utilisent avec `inline-size` , mais aussi comme taille de piste : `grid-template-columns: max-content 1fr;` .

Étape 6 — La grille de cartes « RAM » (≈ 10 min)

Repeat, **A**uto-fit, **M**inmax : des cartes d'au moins `16rem` , autant de colonnes que la largeur le permet, **sans aucune media query**.

1. Écrivez `grid-template-columns` sur `.cartes` .
2. Testez à 320 px : y a-t-il un défilement horizontal ? Corrigez-le avec `min()` .
3. Remplacez `auto-fit` par `auto-fill` , élargissez la fenêtre au maximum (ou zoomez à 50 %) et observez la différence. Gardez la version qui étire les cartes.

Indice 1 — la formule

`repeat(auto-fit, minmax(16rem, 1fr))` : chaque colonne fait au moins 16rem, et les colonnes se partagent le reste à parts égales.

Indice 2 — le débordement à 320 px

16rem = 256 px. Avec les marges, ça ne rentre plus dans 320 px : la colonne déborde. `minmax(min(100%, 16rem), 1fr)` : la colonne ne demande jamais plus que 100 % de la place disponible.

Étape 7 — `subgrid` : des cartes parfaitement alignées (≈ 12 min)

Observez la 1re rangée de cartes : les titres n'ont pas la même longueur, donc les extraits et les dates ne sont pas à la même hauteur d'une carte à l'autre. C'est ce défaut d'alignement que vous allez corriger.

Chaque `.carte` a **5 enfants** : visuel, rubrique, titre, extrait, date.

1. Faites de chaque carte une grille qui **occupe 5 lignes** de `.cartes`…
2. … et qui **adopte ces lignes** au lieu de créer les siennes.
3. Réduisez l'écart **interne** des cartes (elles héritent sinon des `2.5rem` de `.cartes`).
4. Collez les dates (`.carte__meta`) en bas de leur ligne.

Vérifiez avec le badge `subgrid` des DevTools : les lignes horizontales doivent traverser toutes les cartes d'une rangée.

Indice 1 — les deux déclarations clés

```
grid-row: span 5;            /* la carte occupe 5 lignes du parent */
grid-template-rows: subgrid; /* et utilise CES lignes pour ses enfants */
```

Comme `.cartes` n'a pas de `grid-template-rows`, ces 5 lignes par rangée sont créées automatiquement. Leur hauteur est celle du **plus grand** élément de la ligne, toutes cartes confondues.

Indice 2 — pourquoi pas Flexbox ?

Avec `display: flex; flex-direction: column;` et `margin-block-start: auto` sur la date, chaque carte aligne ses dates **en bas**, mais les **titres** et les **extraits** restent désalignés : chaque carte ignore la hauteur des autres. Seul subgrid fait partager les lignes entre cartes sœurs.

Exercice d'approfondissement (facultatif) — La mosaïque « Instantanés du campus »

La galerie contient 7 tuiles : 2 `.tuile--large` (2 colonnes) et 1 `.tuile--haute` (2 lignes).

1. 2 colonnes sur mobile, **5 colonnes** à partir de `48rem`.
2. Des lignes **créées automatiquement** de `8rem` (`10rem` sur grand écran).
3. Les grandes tuiles occupent 2 colonnes, la haute 2 lignes.
4. **Aucun trou** dans la mosaïque, ni sur mobile ni sur ordinateur, sans changer l'ordre du HTML.

5. Expliquez à l'enseignant **pourquoi 5 colonnes**, et ce que l'option qui comble les trous change pour un utilisateur au clavier.

> **Indice**
>
> `grid-auto-rows`, `grid-column: span 2`, `grid-row: span 2`, et une valeur de `grid-auto-flow` qui autorise le navigateur à remonter un élément dans un trou laissé plus haut. Comptez les cellules : 2 × 2 + 1 × 2 + 4 × 1 = ?

Critères de réussite

Auto-évaluez-vous avec cette liste ; l'enseignant la vérifie en une minute, fenêtre à 1280 px puis réduite à 320 px (mode responsive des DevTools) :

- ☐ **Masthead** : logo, liens et bouton sur une ligne à 1280 px, le bouton collé à droite. À 320 px tout passe à la ligne, sans débordement.
- ☐ **Gabarit** : `.page` utilise des **lignes nommées** (visibles dans les DevTools). Le fond de la citation rouge et du pied de page va d'un bord à l'autre, leur texte reste aligné sur la colonne de contenu.
- ☐ **Une** : `grid-template-areas` avec 3 dessins (mobile, 48rem, 64rem). À 1280 px, l'encadré noir est à droite du titre et du visuel.
- ☐ **Visuels** : plus de `min-block-size: 6rem`, les formats viennent d'`aspect-ratio`.
- ☐ **Intrinsèque** : étiquettes à la taille de leur texte, sommaire en `max-content 1fr` avec les numéros alignés, chiffres de l'encadré à droite.
- ☐ **Cartes** : `repeat(auto-fit, minmax(min(100%, 16rem), 1fr))`. Dans une même rangée, titres, extraits et dates commencent à la même hauteur (`subgrid`).
- ☐ **320 px** : aucun défilement horizontal sur toute la page.

Exercice d'approfondissement (facultatif) : réussi si la mosaïque est sans trou à 375 px et à 1280 px et si l'explication orale est correcte.

Questions de réflexion (à l'oral, lors de la vérification par l'enseignant)

1. `auto-fit` ou `auto-fill` : avec seulement **deux cartes** dans un écran de 1280 px, que voit-on dans chaque cas ? Dans quel cas préféreriez-vous `auto-fill` ?
2. Vous avez utilisé `subgrid` **deux fois**, pour des raisons différentes : sur `<main>` et les bandeaux, puis sur les cartes. Quel problème chacun résout-il ? Comment faisait-on avant subgrid, et qu'est-ce que c'était moins bien ?
3. `grid-template-areas` permet de placer visuellement l'encadré « En chiffres » **avant** le texte de l'article alors qu'il est **après** dans le HTML. Quel est le risque pour un utilisateur de lecteur d'écran ou de clavier ? Où est la limite entre « réorganiser pour la mise en page » et « casser l'ordre de lecture » ?